

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 13 | Trees and Binary Trees: Terminology, Representation and Traversals

Week 13 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 13 at a glance

Item	Details
Lab title	Trees and Binary Trees: Terminology, Representation and Traversals
Week	13 of 16
Prerequisites	Lab 06 (Recursion), Lab 10 (Stacks), Lab 11 (Queues)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab13_Instructor.ipynb , Lab13_Student.ipynb
Next lab	Lab 14: Binary Search Trees

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain why trees are **hierarchical (non-linear)** data structures and where they are used.
2. Use tree terminology correctly: **root, parent, child, leaf, sibling, edge, depth, height, level, subtree, degree**.
3. Identify **full, complete, perfect, balanced and degenerate** binary trees.
4. Represent a binary tree with **linked nodes** and with an **array** (index formulas).
5. Implement **preorder, inorder, postorder** (recursive and iterative) and **level-order** traversals.
6. Write recursive functions on trees: size, height, leaves, sum, mirror.
7. Build and evaluate an **expression tree**.

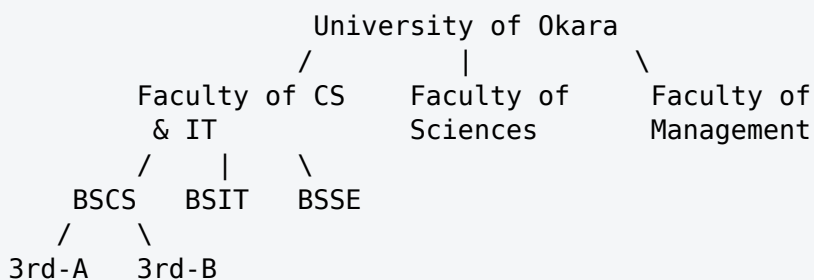
Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-1	Explain tree terminology and binary tree types	Understand (C2)
CLO-2	Implement binary trees and traversals	Apply (C3)
CLO-3	Analyse recursive tree algorithms	Analyse (C4)

2. Motivation: Data with a Hierarchy

Arrays, linked lists, stacks and queues are **linear**: every element has one before and one after. But much real data is **hierarchical**: a university has faculties, which have departments, which have sections. A **tree** represents such “one-to-many” relationships naturally.

A real hierarchy is a tree



2.1 Trees in Daily Life

Example	What forms the tree
Folders on your laptop (C: → Users → Documents ...)	Folders contain folders and files
Family tree (shajra)	Ancestors and descendants
Organisation chart	VC → Deans → HODs → Teachers
Cricket tournament knock-out bracket	Winners move up to the final (a binary tree!)
Website menus	Menu → sub-menu → page
Book contents	Chapters → sections → sub-sections

2.2 Trees in Industry

- **File systems** and the **HTML DOM** (every web page) are trees.
- **Databases** use B-trees and B+ trees for indexes; MySQL, PostgreSQL and Oracle all depend on them.
- **Compilers** turn your code into an **Abstract Syntax Tree** before running it.
- **Compression** (ZIP, JPEG) uses Huffman trees; **routing** and **auto-complete** use tries.
- **JSON and XML** data, used by every web API, are trees.

2.3 Trees in Research

- **Machine learning:** Decision Trees, Random Forests and Gradient Boosting (XGBoost, LightGBM) are among the most widely used models on tabular data.
- **AI search:** game trees and Monte Carlo Tree Search (AlphaGo) explore moves as trees.
- **NLP:** parse trees show the grammatical structure of sentences.
- **GIS & Remote Sensing:** quadtrees, k-d trees and R-trees index spatial data for fast map queries.
- **Biology:** phylogenetic trees show evolutionary relationships.

2.4 Why It Matters in 2026–2030

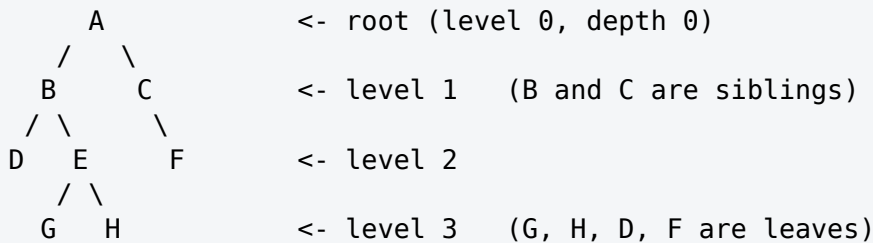


From data structures to AI models

Trees connect DSA directly to AI: a Decision Tree is literally a binary tree where each internal node asks a question about the data. Tree-based models still win many real-world data competitions. Understanding traversal and recursion on trees is the foundation for BSTs, heaps, tries, graph algorithms and these models.

3. Part A: Tree Terminology

Example tree used in this lab



Term	Meaning	In the example
Node	An element of the tree	A, B, ... H (8 nodes)
Edge	A link between parent and child	7 edges (always $n - 1$)
Root	The top node with no parent	A
Parent / Child	A node directly above / below another	B is parent of D and E
Siblings	Nodes with the same parent	D and E; G and H
Leaf	A node with no children	D, G, H, F
Internal node	A node with at least one child	A, B, C, E
Degree	Number of children of a node	$\text{deg}(B) = 2$, $\text{deg}(C) = 1$
Depth of a node	Edges from the root to the node	$\text{depth}(E) = 2$
Height of a node	Edges on the longest path down to a leaf	$\text{height}(B) = 2$
Height of the tree	Height of the root	3
Subtree	A node and all its descendants	B, D, E, G, H
Ancestor / Descendant	Nodes on the path up / below	A, B, E are ancestors of G

Some books define height in terms of nodes instead of edges (so a single node has height 1). In this manual, height counts edges: an empty tree has height -1 and a single node has height 0.

4. Part B: Binary Trees

A **binary tree** is a tree where every node has **at most two children**, called the **left child** and the **right child**.

Type	Definition	Example use
Full (strict)	Every node has 0 or 2 children	Expression trees, Huffman trees
Complete	All levels full except possibly the last, which is filled left to right	Heaps (Lab 15)
Perfect	All internal nodes have 2 children and all leaves are at the same level	Tournament brackets
Balanced	Left and right subtree heights differ by at most 1 at every node	AVL / Red-Black trees
Degenerate (skewed)	Every node has only one child; behaves like a linked list	Worst case for BSTs

4.1 Important Properties

- Maximum nodes at level $k = 2^k$.
- Maximum nodes in a binary tree of height $h = 2^{h+1} - 1$ (a perfect tree).
- Minimum height of a binary tree with n nodes = $\lceil \log_2 n \rceil$. This is why balanced trees give $O(\log n)$ operations.
- In a full binary tree, number of leaves = number of internal nodes + 1.

4.2 Representation 1: Linked Nodes

```
class TreeNode:
    def __init__(self, data, left=None, right=None):
        self.data = data
        self.left = left
        self.right = right

def build_tree(values):
    """Build a binary tree from a level-order list; None means 'no
    node'."""
    if not values or values[0] is None:
        return None
    from collections import deque
    root = TreeNode(values[0])
    q, i = deque([root]), 1
    while q and i < len(values):
        node = q.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i]); q.append(node.left)
        i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i]); q.append(node.right)
        i += 1
    return root

# Build the example tree by hand
G, H = TreeNode("G"), TreeNode("H")
D, E = TreeNode("D"), TreeNode("E", G, H)
F = TreeNode("F")
B_, C_ = TreeNode("B", D, E), TreeNode("C", None, F)
root = TreeNode("A", B_, C_)

def show(node, indent="", label="root"):
    """Print the tree sideways: right subtree on top."""
    if node is None:
        return
    show(node.right, indent + "      ", "R")
    print(f"{indent}{label}: {node.data}")
    show(node.left, indent + "      ", "L")

show(root)
```

4.3 Representation 2: Array (for Complete Trees)

Store nodes level by level in an array. For the node at index **i** (0-based): left child at **2i + 1**, right child at **2i + 2**, parent at **(i - 1) // 2**. No pointers needed; this is exactly how heaps are stored in Lab 15.

```
tree = ["A", "B", "C", "D", "E", "F", "G"]    # a complete tree
i = 1                                         # node "B"
print("node:", tree[i], "| left:", tree[2 * i + 1], "| right:", tree[2 * i
+ 2],
      "| parent:", tree[(i - 1) // 2])
```



5. Part C: Tree Traversals

A **traversal** visits every node exactly once. Depth-first traversals differ only in **when** the current node is visited relative to its subtrees:

Traversal	Order	Result on the example tree	Typical use
Preorder	Node → Left → Right	A B D E G H C F	Copy a tree, print folder structure
Inorder	Left → Node → Right	D B G E H A C F	Sorted order in a BST (Lab 14)
Postorder	Left → Right → Node	D G H E B F C A	Delete a tree, compute folder sizes
Level order	Level by level, left to right	A B C D E F G H	Shortest paths, printing by levels



Memory trick

The name tells you where the **node** goes: **Pre** = node first, **In** = node in the middle, **Post** = node last. Left always comes before right.

5.1 Recursive Depth-First Traversals

```
def preorder(node, out):
    if node:
        out.append(node.data)          # visit
        preorder(node.left, out)
        preorder(node.right, out)
    return out

def inorder(node, out):
    if node:
        inorder(node.left, out)
        out.append(node.data)          # visit
        inorder(node.right, out)
    return out

def postorder(node, out):
    if node:
        postorder(node.left, out)
        postorder(node.right, out)
        out.append(node.data)          # visit
    return out

print("pre :", " ".join(preorder(root, [])))
print("in  :", " ".join(inorder(root, [])))
print("post:", " ".join(postorder(root, [])))
```

5.2 Level-Order Traversal with a Queue (Breadth-First)

```
from collections import deque

def level_order(root):
    if root is None:
        return []
    out, q = [], deque([root])
    while q:
        node = q.popleft()
        out.append(node.data)
        if node.left:
            q.append(node.left)
        if node.right:
            q.append(node.right)
    return out

print("level:", " ".join(level_order(root)))
```

5.3 Iterative Preorder with a Stack

Recursion uses the call stack; we can use our own stack instead. Push the right child **before** the left child so that the left one is processed first.

```
def preorder_iter(root):
    if root is None:
        return []
    out, stack = [], [root]
    while stack:
        node = stack.pop()
        out.append(node.data)
        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)
    return out

print("pre (stack):", " ".join(preorder_iter(root)))
```

Every traversal visits each node once, so all are **$O(n)$** time. Space is **$O(h)$** for DFS (the recursion/stack depth) and **$O(w)$** for BFS, where w is the maximum width of the tree.



6. Part D: Recursive Thinking on Trees

Almost every tree problem follows one pattern: **solve for the left subtree, solve for the right subtree, combine with the current node**. The base case is the empty tree (**None**).

```
def size(node):
    return 0 if node is None else 1 + size(node.left) + size(node.right)

def height(node):
    if node is None:
        return -1 # empty tree
    return 1 + max(height(node.left), height(node.right))

def count_leaves(node):
    if node is None:
        return 0
    if node.left is None and node.right is None:
        return 1
    return count_leaves(node.left) + count_leaves(node.right)

print("size:", size(root), "| height:", height(root), "| leaves:",
      count_leaves(root))
```



7. Part E: Expression Trees

An arithmetic expression can be stored as a binary tree: **leaves are numbers** and **internal nodes are operators**. Inorder traversal (with brackets) gives the infix form,

postorder gives the postfix form (Lab 10), and evaluating is a postorder computation.

Expression tree



```

expr = TreeNode("*", TreeNode("+", TreeNode(3), TreeNode(5)),
                  TreeNode("-", TreeNode(8), TreeNode(2)))

def evaluate(node):
    if node.left is None and node.right is None:
        return node.data # a number
    a, b = evaluate(node.left), evaluate(node.right)
    return {"+": a + b, "-": a - b, "*": a * b, "/": a / b}[node.data]

def to_infix(node):
    if node.left is None and node.right is None:
        return str(node.data)
    return f"({to_infix(node.left)} {node.data} {to_infix(node.right)})"

print(to_infix(expr), "=", evaluate(expr))
print("postfix:", " ".join(str(x) for x in postorder(expr, [])))
  
```

8. C++ Corner

C++

```
#include <iostream>
#include <queue>
#include <algorithm>
using namespace std;

struct Node {
    char data; Node* left; Node* right;
    Node(char d, Node* l = nullptr, Node* r = nullptr) : data(d), left(l), right(r) {}
};

void inorder(Node* n) { if (!n) return; inorder(n->left); cout << n->data << " "; inorder(n->right); }
int height(Node* n) { return n ? 1 + max(height(n->left), height(n->right)) : -1; }

void levelOrder(Node* root) {
    queue<Node*> q; if (root) q.push(root);
    while (!q.empty()) {
        Node* n = q.front(); q.pop();
        cout << n->data << " ";
        if (n->left) q.push(n->left);
        if (n->right) q.push(n->right);
    }
}

void destroy(Node* n) { if (!n) return; destroy(n->left); destroy(n->right); delete n; } // postorder!

int main() {
    Node* root = new Node('A', new Node('B', new Node('D'),
                                           new Node('E', new Node('G'), new Node('H'))),
                           new Node('C', nullptr, new Node('F')));
    inorder(root); cout << endl; // D B G E H A C F
    levelOrder(root); cout << endl; // A B C D E F G H
    cout << height(root) << endl; // 3
    destroy(root);
}
```

9. Common Mistakes

Mistake	Effect	Fix
Forgetting the None base case	AttributeError on None.left	Start every recursive function with if node is None
Mixing up height in nodes vs edges	Off-by-one answers	State your definition; empty tree = -1 here
Pushing left before right in iterative preorder	Right subtree printed first	Push right first, then left
Using a list with pop(0) for BFS	O(n ²) for big trees	Use collections.deque

Mistake	Effect	Fix
Returning the result of only one subtree	Wrong counts / sums	Combine both left and right results

10. Lab Tasks

Solve all tasks in `Lab13_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Sum, Maximum and Leaf Values

The notebook provides `TreeNode` and `build_tree(level_order_list)`. Write recursive functions `tree_sum(root)`, `tree_max(root)` (return `None` for an empty tree) and `leaf_values(root)` (left-to-right list of leaf data).

```
build_tree([5, 3, 8, 1, 4, None, 9]) → sum 30, max 9, leaves [1, 4, 9]
```

Task 2 Mirror (Invert) a Binary Tree

Write `mirror(root)` that swaps the left and right child of **every** node and returns the root. Check your answer with level-order traversal.

```
[4, 2, 7, 1, 3, 6, 9] → level order [4, 7, 2, 9, 6, 3, 1]
```

Task 3 Same Tree and Symmetric Tree

Write `same_tree(a, b)` (same shape and same values) and `is_symmetric(root)` (the tree is a mirror image of itself around its centre). Hint: a tree is symmetric if its left subtree is the mirror of its right subtree; write a helper `is_mirror(x, y)`.

```
[1, 2, 2, 3, 4, 4, 3] → symmetric
[1, 2, 2, None, 3, None, 3] → not symmetric
```

Task 4 Level-by-Level Report

Write `levels(root)` that returns a list of lists: the values at each level, from top to bottom. Then write `right_view(root)`: the last value of each level (what you see when looking at the tree from the right side).

```
[3, 9, 20, None, None, 15, 7] → levels [[3], [9, 20], [15, 7]], right view [3, 20, 7]
```

Task 5 Iterative Inorder Traversal

Write `inorder_iter(root)` **without recursion**, using an explicit stack. Hint: go left as far as possible pushing nodes; pop a node, visit it, then move to its right child.

```
[1, None, 2, 3] → [1, 3, 2]
```

Task 6 Diameter of a Binary Tree

The **diameter** is the number of edges on the longest path between **any two nodes** (the path may or may not pass through the root). A naive solution computes height at every node ($O(n^2)$).

Write an **$O(n)$** solution with a helper that returns the height of a subtree while updating the best diameter seen so far.

```
[1, 2, 3, 4, 5] → 3    (path 4 → 2 → 1 → 3)
```

Task 7 Rebuild a Tree from Traversals

Given the **preorder** and **inorder** traversals of a binary tree with unique values, rebuild the tree and return its root.

Key idea: the first preorder value is the root; its position in inorder splits the left and right subtrees. Use a dictionary of inorder positions for $O(n)$.

```
preorder [3, 9, 20, 15, 7], inorder [9, 3, 15, 20, 7] → tree [3, 9, 20, None,
None, 15, 7]
```



11. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub **DSA-Labs/Lab13** folder.

#	Problem	Level	Concept practised
144	Binary Tree Preorder Traversal	Easy	DFS
94	Binary Tree Inorder Traversal	Easy	DFS / stack (Task 5)
145	Binary Tree Postorder Traversal	Easy	DFS
104	Maximum Depth of Binary Tree	Easy	Height
226	Invert Binary Tree	Easy	Mirror (Task 2)
101	Symmetric Tree	Easy	Mirror check (Task 3)
102	Binary Tree Level Order Traversal	Medium	BFS (Task 4)
543	Diameter of Binary Tree	Easy	Height + global best (Task 6)

#	Problem	Level	Concept practised
105	Construct Binary Tree from Preorder and Inorder Traversal	Medium	Divide & conquer (Task 7)



12. Home Task

1. Finish all tasks and push `Lab13_Student.ipynb` to GitHub.
2. Write `is_complete(root)` (use BFS: once you see a missing child, no later node may have children) and `is_balanced(root)` in $O(n)$.
3. Write iterative **postorder** traversal using two stacks.
4. **Mini-project: Folder Size Analyser.** Represent a folder hierarchy as a general tree (each node has a list of children and a file size). Use a postorder traversal to compute the total size of every folder, print the tree with indentation, and list the 3 largest folders. Then try it on a real folder with `os.scandir`.
5. Solve at least 3 LeetCode problems from the list.

? 13. Viva Questions

1. Why is a tree called a non-linear data structure?
2. Define root, leaf, sibling, depth and height.
3. How many edges does a tree with n nodes have?
4. Differentiate between full, complete and perfect binary trees.
5. What is the maximum number of nodes in a binary tree of height h ?
6. In the array representation, where are the children of index i ?
7. Write the preorder, inorder and postorder of a small tree you draw.
8. Which data structure does level-order traversal use? Which does iterative DFS use?
9. What is the time and space complexity of tree traversals?
10. What is an expression tree? Which traversal evaluates it?



14. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



15. References and Further Learning

- Binary tree animations: visualgo.net/en/bst
- scikit-learn Decision Trees: scikit-learn.org/stable/modules/tree.html
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.